

TryHackMe - Sysmon

Now that Sysmon is started with the configuration file we want to use, we can look at the Event Viewer to monitor events. The event log is located under *Applications and Services Logs/Microsoft/Windows/Sysmon/Operational*

Event ID 1: Process Creation

This event will look for any processes that have been created. You can use this to look for known suspicious processes or processes with typos that would be considered an anomaly. This event will use the CommandLine and Image XML tags.

```
<RuleGroup name="" groupRelation="or"> <ProcessCreate onmatch="exclude">  
<CommandLine condition="is">C:\Windows\system32\svchost.exe -k appmodel -p -s  
camsvc</CommandLine> </ProcessCreate> </RuleGroup>
```

The above code snippet is specifying the Event ID to pull from as well as what condition to look for. In this case, it is excluding the svchost.exe process from the event logs.

Event ID 3: Network Connection

The network connection event will look for events that occur remotely. This will include files and sources of suspicious binaries as well as opened ports. This event will use the Image and DestinationPort XML tags.

```
<RuleGroup name="" groupRelation="or"> <NetworkConnect onmatch="include"> <Image  
condition="image">nmap.exe</Image> <DestinationPort name="Alert, Metasploit"  
condition="is">4444</DestinationPort> </NetworkConnect> </RuleGroup>
```

The above code snippet includes two ways to identify suspicious network connection activity. The first way will identify files transmitted over open ports. In this case, we are specifically looking for nmap.exe which will then be reflected within the event logs. The second method identifies open ports and specifically port 4444 which is commonly used with Metasploit. If the condition is met an event will be created and ideally trigger an alert for the SOC to further investigate.

Event ID 7: Image Loaded

This event will look for DLLs loaded by processes, which is useful when hunting for DLL Injection and DLL Hijacking attacks. It is recommended to exercise caution when using this Event ID as it causes a high system load. This event will use the Image, Signed, ImageLoaded, and Signature XML tags.

```
<RuleGroup name="" groupRelation="or"> <ImageLoad onmatch="include">  
<ImageLoaded condition="contains">\Temp\</ImageLoaded> </ImageLoad> </RuleGroup>
```

The above code snippet will look for any DLLs that have been loaded within the \Temp\ directory. If a DLL is loaded within this directory it can be considered an anomaly and should be further investigated.

Event ID 8: CreateRemoteThread

The CreateRemoteThread Event ID will monitor for processes injecting code into other processes. The CreateRemoteThread function is used for legitimate tasks and applications. However, it could be used by malware to hide malicious activity. This event will use the SourceImage, TargetImage, StartAddress, and StartFunction XML tags.

```
<RuleGroup name="" groupRelation="or"> <CreateRemoteThread onmatch="include">  
<StartAddress name="Alert,Cobalt Strike" condition="end with">0B80</StartAddress>  
  <SourceImage condition="contains">\</SourceImage> </CreateRemoteThread>  
</RuleGroup>
```

The above code snippet shows two ways of monitoring for CreateRemoteThread. The first method will look at the memory address for a specific ending condition which could be an indicator of a Cobalt Strike beacon. The second method will look for injected processes that do not have a parent process. This should be considered an anomaly and require further investigation.

Event ID 11: File Created

This event ID is will log events when files are created or overwritten the endpoint. This could be used to identify file names and signatures of files that are written to disk. This event uses TargetFilename XML tags.

```
<RuleGroup name="" groupRelation="or"> <FileCreate onmatch="include">  
<TargetFilename name="Alert,Ransomware"  
condition="contains">HELP_TO_SAVE_FILES</TargetFilename> </FileCreate>  
</RuleGroup>
```

The above code snippet is an example of a ransomware event monitor. This is just one example of a variety of different ways you can utilize Event ID 11.

Event ID 12 / 13 / 14: Registry Event

This event looks for changes or modifications to the registry. Malicious activity from the registry can include persistence and credential abuse. This event uses TargetObject XML tags.

```
<RuleGroup name="" groupRelation="or"> <RegistryEvent onmatch="include">
<TargetObject name="T1484"
condition="contains">Windows\System\Scripts</TargetObject> </RegistryEvent>
</RuleGroup>
```

The above code snippet will look for registry objects that are in the *"Windows\System\Scripts"* directory as this is a common directory for adversaries to place scripts to establish persistence.

Event ID 15: FileCreateStreamHash

This event will look for any files created in an alternate data stream. This is a common technique used by adversaries to hide malware. This event uses TargetFilename XML tags.

```
<RuleGroup name="" groupRelation="or"> <FileCreateStreamHash onmatch="include">
<TargetFilename condition="end with">.hta</TargetFilename> </FileCreateStreamHash>
</RuleGroup>
```

The above code snippet will look for files with the .hta extension that have been placed within an alternate data stream.

Event ID 22: DNS Event

This event will log all DNS queries and events for analysis. The most common way to deal with these events is to exclude all trusted domains that you know will be very common "noise" in your environment. Once you get rid of the noise you can then look for DNS anomalies. This event uses QueryName XML tags.

```
<RuleGroup name="" groupRelation="or"> <DnsQuery onmatch="exclude"> <QueryName
condition="end with">.microsoft.com</QueryName> </DnsQuery> </RuleGroup>
```

The above code snippet will get exclude any DNS events with the .microsoft.com query. This will get rid of the noise that you see within the environment.

Filter by Event ID: `*/System/EventID=<ID>`

Filter by XML Attribute/Name: `*/EventData/Data[@Name="<XML Attribute/Name>"]`

Filter by Event Data: `*/EventData/Data=<Data>`

We can put these filters together with various attributes and data to get the most control out of our logs. Look below for an example of using `Get-WinEvent` to look for network connections coming from port 4444.

```
Get-WinEvent -Path <Path to Log> -FilterXPath '*/System/EventID=3 and
*/EventData/Data[@Name="DestinationPort"] and */EventData/Data=4444'
```

How many event ID 3 events are in C:\Users\THM-Analyst\Desktop\Scenarios\Practice\Filtering.evtx?

```
Get-WinEvent -Path C:\Users\THM-Analyst\Desktop\Scenarios\Practice\Filtering.evtx
-FilterXPath '*/System/EventID=3' | Measure-Object -Line
```

What is the UTC time created of the first network event in C:\Users\THM-Analyst\Desktop\Scenarios\Practice\Filtering.evtx?

2021-01-06 01:35:50.464